

To answer your first question: **No, the HLK-PM01 (5V 3W) does not give the full wattage needed when the siren sounds.**

The HLK-PM01 only supplies 600mA (3 Watts). The ESP32 can draw up to 300-500mA during WiFi transmission, the TP4056 can draw up to 1000mA (1A) while charging a depleted battery, and a high-decibel siren can pull another 300-500mA. Running all of this on a 3W converter will cause a brownout, crashing the ESP32 or failing to trigger the siren.

To achieve your goal where the system runs perfectly on AC without touching the battery, and switches instantly to battery power only when the grid fails, we must introduce an **HLK-10M05 (5V 10W / 2A)** and a **Solid-State Diode OR-ing Power Path**. This guarantees zero battery drain while the AC is on, keeps the battery fully charged, and provides instant switchover without rebooting the ESP32.

Here is your final, upgraded Master Node cheat sheet.

## 1. The Upgraded Bill of Materials (BOM)

### Power & True UPS Matrix (Upgraded)

- **HLK-10M05 (5V 10W / 2A):** Upgraded AC-DC converter. Replaces the 3W module to provide enough current for the ESP32, siren, and battery charger simultaneously.
- **2x 1N5822 Schottky Diodes (3A):** The secret to the instant UPS switchover. These route power automatically without back-feeding or draining the battery when AC is active.
- **TP4056 Module (with DW01 protection):** Manages the 18650 battery charging.
- **18650 Li-Ion Battery:** Your backup power.
- **MT3608 Boost Converter:** Steps the 3.7V battery up to 5V.

### Logic, Actuation & Telemetry (Ultra-Low Power)

- **ESP32 WROOM-32 DevKit:** Master Node.
- **5V Relay Module:** Switches the siren ON/OFF.
- **High-Decibel 5V/12V Siren:** Running directly off the shared 5V main rail.
- **2x 1MΩ Resistors + 1x 0.1μF Ceramic Capacitor:** Cuts the battery voltage in half so the ESP32 can read it safely. Connecting this directly to the battery draws an ultra-low ~2.1 microamps, keeping the battery safe from phantom drain.

## 2. True UPS Power Path & Unified 5V Connection Diagram

*How it works: The MT3608 is set to exactly 4.8V, while the AC-DC converter outputs 5.0V.*

*Because electricity flows from the highest voltage source, the AC-DC module handles 100% of the load through Diode 1. Diode 2 remains "closed". The moment AC power drops, the voltage dips, Diode 2 instantly opens, and the battery takes over with zero interruption.*

```
=====
=====
ZONE A: TRUE UPS POWER GENERATION & BATTERY CHARGING
=====
```

=====

[ 220V AC Mains ] —> [ HLK-10M05 (5V 10W) ]

(1) CHARGING PATH:

[ HLK-10M05 5V (V+) ] —————> [ TP4056 IN+ ]

[ HLK-10M05 GND (V-) ] —————> [ TP4056 IN- ]

(2) BATTERY PATH:

[ 18650 Battery (+) ] —————> [ TP4056 B+ ]

|  
[ 1M $\Omega$  Resistor ]

| —————> [ ESP32 GPIO 34 ] (Battery Monitor Pin)

|  
[ 1M $\Omega$  Resistor ] + [ 0.1 $\mu$ F Cap ]

|  
[ 18650 Battery (-) ] —————> [ TP4056 B- ]

(3) BOOST PATH (Set MT3608 to exactly 4.8V):

[ TP4056 OUT+ ] —————> [ MT3608 IN+ ]

[ TP4056 OUT- ] —————> [ MT3608 IN- ]

=====

=====

ZONE B: INSTANT LOAD-SHARING & MAIN 5V RAIL (DIODE OR-ING)

=====

=====

[ HLK-10M05 5V (V+) ] ———>| (Diode 1) ———|

|  
|  
|

| —————> [ MAIN 5V RAIL (VCC) ]

[ MT3608 OUT+ (4.8V) ] ———>| (Diode 2) ———|

\*Connect all Grounds together:\*

[ HLK-10M05 GND ] ———| [ MT3608 OUT- ] ———|> [ MAIN GND RAIL ]

|

(TP4056 IN-)

|

(TP4056 OUT-)

=====

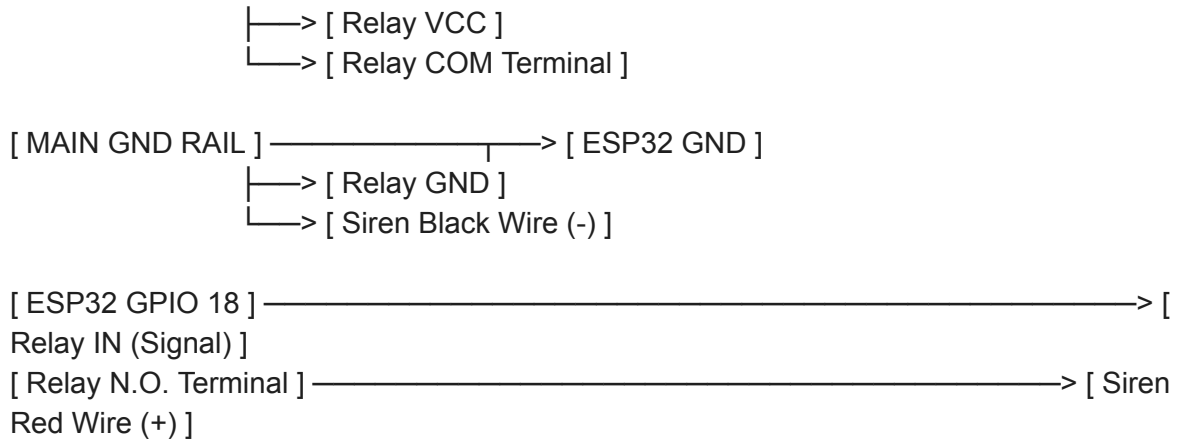
=====

ZONE C: LOGIC & SIREN ACTUATION

=====

=====

[ MAIN 5V RAIL (VCC) ] ———|> [ ESP32 5V (VIN) ]



### 3. Master Node Firmware Updates (Adding Battery Telemetry)

To make your Master Node report its battery percentage to Home Assistant, insert these blocks into your existing master-node.ino code.

**Step A: Define the Battery Pin (Add at the very top of your code)**

```

// --- ADD THIS TO YOUR PIN ASSIGNMENTS ---
#define BAT_ADC_PIN 34 // ESP32 WROOM Analog Pin for Battery Monitoring
uint8_t masterBatteryPct = 100;

```

**Step B: Add the Voltage Math Function (Add above setup())**

```

// --- ADD THIS FUNCTION TO CALCULATE PERCENTAGE ---
uint8_t calculateMasterBattery {
  uint32_t adc_mv = analogReadMilliVolts(BAT_ADC_PIN);

  // Multiply by 2.0 because the 1M/1M resistor divider cuts voltage in half
  float battery_voltage = (adc_mv * 2.0) / 1000.0;
  Serial.printf("[UPS BATTERY] Voltage: %.2fV\n", battery_voltage);

  // Map 18650 Lithium discharge curve (4.2V = 100%, 3.3V = 0% dead)
  int pct = map(battery_voltage * 100, 330, 420, 0, 100);
  pct = constrain(pct, 0, 100);
}

```

```

    return (uint8_t)pct;
}

```

### Step C: Update your existing publishHAState() function

```

void publishHAState {
    if (!mqttClient.connected()) return;

    mqttClient.publish("home/master_node/siren/state", emergencyAlarmActive ? "ON" : "OFF",
true);
    mqttClient.publish("home/master_node/ap_mode/state", isEmergencyApActive ? "ON" :
"OFF", true);
    mqttClient.publish("home/alarms/master_siren", emergencyAlarmActive ? "TRIGGERED" :
"SAFE", true);
    mqttClient.publish("home/alarms/reason", emergencyAlarmActive ? "Manual Alarm Active" :
"System Armed - All Safe", true);
    mqttClient.publish("home/master_node/ap_mode/status", isEmergencyApActive ? "AP
ACTIVE: 192.168.4.1" : "AP STANDBY (Router Online)", true);

    // --- ADD THESE 3 LINES TO PUBLISH THE BATTERY TO HA ---
    char batStr[8];
    snprintf(batStr, sizeof(batStr), "%d", masterBatteryPct);
    mqttClient.publish("home/master_node/battery", batStr, true);
}

```

### Step D: Update your loop() to read the battery regularly

```

void loop {
    // Your existing BOOT button factory reset code...

    // EMERGENCY AP SUPERVISOR (Checks Router state every 60 seconds)
    if (millis() - lastWifiCheckTime >= WIFI_CHECK_INTERVAL_MS) {
        lastWifiCheckTime = millis();
    }
}

```

```

// --- ADD THESE 2 LINES TO UPDATE BATTERY EVERY 60 SECONDS ---
masterBatteryPct = calculateMasterBattery();
publishHASSState();

// Your existing Router Offline/Online AP logic...
}
// Your existing Captive Portal and delay(10) code...
}

```

## 4. Home Assistant Configuration (configuration.yaml)

Add this new sensor so you can track the Master Node's UPS battery health in your dashboard. If the mains power goes out, you will be able to watch this percentage drop over time.

```

mqtt:
  sensor:
    - name: "Master Hub Battery"
      state_topic: "home/master_node/battery"
      unit_of_measurement: "%"
      device_class: battery
      state_class: measurement
      icon: "mdi:battery-charging-high"
    device:
      identifiers: ["redwans_master_node"]
      name: "REDWAN-MASTER-NODE"
      model: "ESP32 WROOM Center Hub"
      manufacturer: "Expressif / Redwan"

```